LAB 13 MVVM architecture, repositories, dependency injection and testing

Unit 5 • 3-hour guided laboratory • CLO-5

Learning goal. Move business logic away from widgets, inject repository dependencies and verify behavior using automated Dart and Flutter widget tests.

Core concepts. Model/View/ViewModel separation, repository interfaces, constructor injection, ChangeNotifier, unit vs widget tests, deterministic fake data.

Before you start. Familiarity with ChangeNotifier and collections. Create separate projects for Task 1 and Task 2 or replace all named files.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab13
cd mad_lab13
```

Architecture overview

View = widget tree and user interactions; ViewModel = UI-facing state and actions; Repository = data access contract; Data source = network, local database or test fake. Dependency injection supplies a specific repository instance to the ViewModel. A ViewModel should not import MaterialApp or navigate using BuildContext unless the application design explicitly assigns it that responsibility.

SUGGESTED DIRECTORY TREE — Task 1

```
lib/
  main.dart                # View + application bootstrap
  data/course_repository.dart   # Contract and implementation
  viewmodel/course_viewmodel.dart  # State and business logic
```

Solved task 1 | Testable course browser using simple MVVM

Problem and objective. Separate course loading from widget rendering. Inject a repository instance through the ViewModel constructor rather than allowing the page to create network/database details.

lib/data/course_repository.dart

```
abstract class CourseRepository {
  Future<List<String>> loadCourses();
}

class DemoCourseRepository implements CourseRepository {
  @override
  Future<List<String>> loadCourses() async {
    await Future<void>.delayed(const Duration(milliseconds: 300));
    return ['Mobile App Development', 'Database Systems',
      'Software Testing', 'Operating Systems'];
  }
}
```

lib/viewmodel/course_viewmodel.dart

```
import 'package:flutter/foundation.dart';
import '../data/course_repository.dart';

class CourseViewModel extends ChangeNotifier {
  final CourseRepository repository;
  CourseViewModel(this.repository);
  List<String> courses = [];
  String? error;
  bool loading = false;

  Future<void> load() async {
    loading = true;
    error = null;
    notifyListeners();
```

```
    try {
      courses = await repository.loadCourses();
    } catch (e) {
      error = 'Unable to load courses: $e';
    } finally {
      loading = false;
      notifyListeners();
    }
  }
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'data/course_repository.dart';
import 'viewmodel/course_viewmodel.dart';

void main() => runApp(const MaterialApp(home: CourseScreen()));

class CourseScreen extends StatefulWidget {
  const CourseScreen({super.key});
  @override
  State<CourseScreen> createState() => _CourseScreenState();
}

class _CourseScreenState extends State<CourseScreen> {
  late final CourseViewModel model;

  @override
  void initState() {
    super.initState();
    model = CourseViewModel(DemoCourseRepository())..load();
  }

  @override
  void dispose() { model.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('MVVM course list')),
    body: AnimatedBuilder(
      animation: model,
      builder: (context, _) {
        if (model.loading) {
          return const Center(child: CircularProgressIndicator());
        }
        if (model.error != null) return Center(child: Text(model.error!));
        return ListView.builder(
          itemCount: model.courses.length,
          itemBuilder: (_, i) => ListTile(title: Text(model.courses[i])),
        );
      },
    ),
  );
}
```

How the solution works. The repository contract isolates data operations; the ViewModel manages loading, data and error state; the View uses an AnimatedBuilder to listen for change notifications. Constructor injection allows replacement of DemoCourseRepository with an API repository or deterministic fake without rewriting the screen.

Expected output. A brief progress spinner transitions into four course rows without any networking.

Check your work. Temporarily throw Exception from DemoCourseRepository and verify the ViewModel reports a user-readable error state.

Solved task 2 | Unit tests + widget test with runnable app

Problem and objective. Practice red/green testing on pure score logic and widget behavior without connecting to external services. The Flutter template includes flutter_test as a dev dependency.

lib/course_math.dart

```
int totalScore({required int assignment,
    required int midterm, required int finalExam}) {
  if (assignment < 0 || assignment > 25 ||
      midterm < 0 || midterm > 25 ||
      finalExam < 0 || finalExam > 50) {
    throw ArgumentError('Marks out of range');
  }
  return assignment + midterm + finalExam;
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'course_math.dart';

void main() => runApp(const MaterialApp(home: ScorePage()));

class ScorePage extends StatefulWidget {
  const ScorePage({super.key});
  @override
  State<ScorePage> createState() => _ScorePageState();
}

class _ScorePageState extends State<ScorePage> {
  int score = 0;
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Score tests')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Score: $score', key: const Key('score_text')),
        FilledButton(
          onPressed: () => setState(() => score = totalScore(
            assignment: 20, midterm: 20, finalExam: 40)),
          child: const Text('Calculate'),
        ),
      ],
    )),
  );
}
```

test/course_math_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/course_math.dart';

void main() {
  test('all maximum marks sum to 100', () {
    expect(totalScore(assignment: 25, midterm: 25,
        finalExam: 50), 100);
  });

  test('out-of-range marks are rejected', () {
    expect(() => totalScore(assignment: 26, midterm: 20,
      finalExam: 40), throwsArgumentError);
  });
}
```

test/widget_test.dart — REPLACE generated starter test

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/main.dart';

void main() {
  testWidgets('calculate button displays 80', (tester) async {
    await tester.pumpWidget(const MaterialApp(home: ScorePage()));
    expect(find.text('Score: 0'), findsOneWidget);
    await tester.tap(find.text('Calculate'));
    await tester.pump();
    expect(find.text('Score: 80'), findsOneWidget);
  });
```

```
}
```

TERMINAL — run checks

```
dart format lib test
flutter analyze
flutter test
flutter run
```

How the solution works. A pure Dart function checks bounds and returns a deterministic result, making it easy to test independently from rendering. The widget test pumps ScorePage, taps the button and asserts visible output. flutter test reports failures before the app is deployed.

Expected output. Two unit tests and one widget test pass; the phone app updates the visible score from 0 to 80.

Check your work. Ensure your flutter create project name really is mad_lab13, because the package imports in test/ match the generated pubspec.yaml name.

Dependency inversion and design reasoning

High-level business rules should depend on contracts, not low-level HTTP/database details. The ViewModel in Task 1 knows only CourseRepository; it does not know whether entries came from an API, Drift or a fake repository. For larger apps, consider a dependency container, typed error objects and immutable view state, but avoid adding complexity without a concrete need.

Scenario-based practice — unsolved

Scenario 1. Hospital queue architecture. Separate queue screen, queue ViewModel, server repository and cache repository. Explain which parts can be unit-tested without an Android phone.

Scenario 2. Fault injection. Your network repository intermittently fails and returns duplicate records. Create fake responses for latency, HTTP failure, empty data and duplicates; define UI states and acceptance tests.

Technical viva — justify your reasoning

1. How does constructor injection reduce coupling?
2. What is the difference between a repository and a ViewModel?
3. Why is a pure function easier to unit-test?
4. What does pumpWidget create, and what does pump rebuild?
5. How would you test a ViewModel that relies on an asynchronous repository?
6. What behavior justifies a mock/fake data source rather than a live backend test?

Submission and self-check

Submit architecture folder tree, tests showing PASS, screenshot of the app and the output of flutter analyze. Assignment checkpoint 5 can use these artifacts.

CSC303 / NOVEMBER 2026 REVISION

LAB 13 EXTENSION | Unit-test the repository/ViewModel boundary

Run this extension in a copy of **Task 1** that still contains `lib/data/course_repository.dart` and `lib/viewmodel/course_viewmodel.dart`. Do not add it to the unrelated score-screen Task 2 replacement.

New file: test/course_viewmodel_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mad_lab13/data/course_repository.dart';
import 'package:mad_lab13/viewmodel/course_viewmodel.dart';

class FakeRepo implements CourseRepository {
  final bool shouldFail;
  FakeRepo({this.shouldFail = false});
  @override
  Future<List<String>> loadCourses() async {
    if (shouldFail) throw StateError('test outage');
    return ['Mobile App Development', 'Operating Systems'];
  }
}

void main() {
  test('loads and reports courses', () async {
    final vm = CourseViewModel(FakeRepo());
    final transitions = <bool>[];
    vm.addListener(() => transitions.add(vm.loading));
    await vm.load();
    expect(transitions, [true, false]);
    expect(vm.courses, hasLength(2));
    expect(vm.error, isNull);
    vm.dispose();
  });
  test('reports repository error without staying busy', () async {
    final vm = CourseViewModel(FakeRepo(shouldFail: true));
    await vm.load();
    expect(vm.loading, isFalse);
    expect(vm.error, contains('test outage'));
    vm.dispose();
  });
}
```

If you created a different package name, change both import prefixes to match `pubspec.yaml`. These tests demonstrate dependency inversion directly: no HTTP request, database, Firebase project or phone is required.

Run and explain

```
dart format lib test
flutter analyze
flutter test test/course_viewmodel_test.dart
```

- Explain the fake repository and why it is deterministic.
- Show error and successful-load state transitions and how the UI listens for notifications.
- For a more advanced submission, add a pending Future to prove the loading indicator remains active while the repository has not completed.